

sAIfer Lab

Joint lab on Safety and Security of AI

# Struttura iterativa while



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI



sAIfer Lab

Joint lab on Safety and Security of AI

# Struttura iterativa while

Perché servono i cicli, sintassi, schemi logici

# Perché servono i cicli

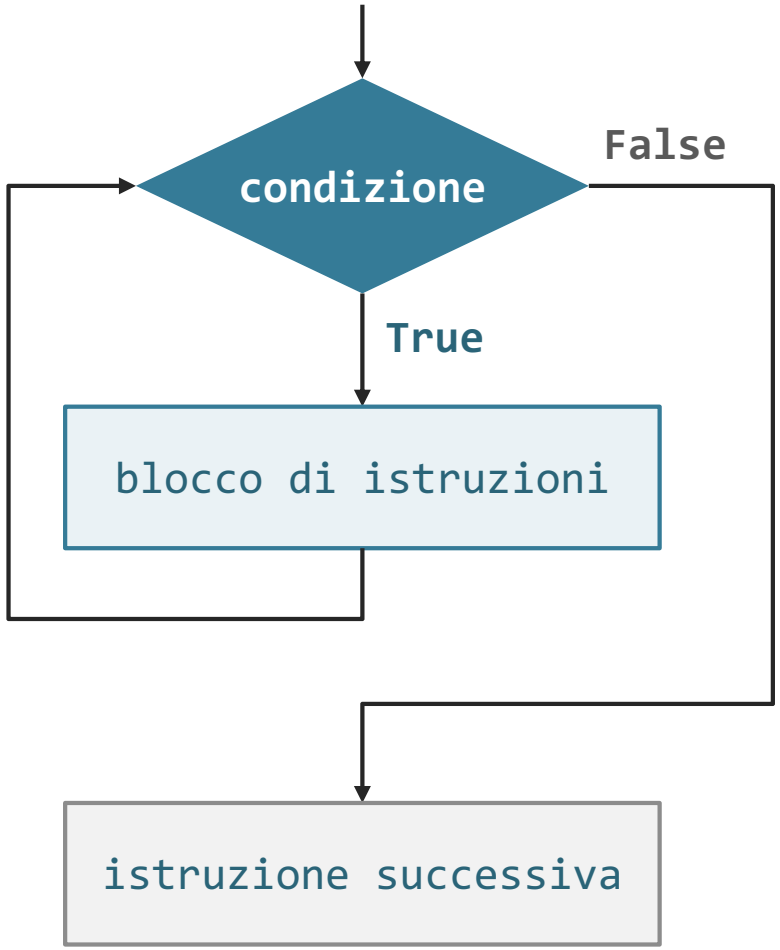
Spesso dobbiamo ripetere più volte le stesse istruzioni

```
# TABELLINE - stampa la tabellina del 3
n_tabellina = 3
i = 1
risultato = n_tabellina * i
print(n_tabellina, '*', i, '=', risultato)

i = 2
risultato = n_tabellina * i
print(n_tabellina, '*', i, '=', risultato)
# ... e così via per i = 3, 4, ...
```

Per stampare i primi 1000 valori questa soluzione è impraticabile

# Il costrutto while risolve il problema




- La `condizione` viene valutata
- Se è vera, viene eseguito il corpo del **while** e la `condizione` viene rivalutata
- Se è falsa, si esce dal ciclo

# while – sintassi semplificata

```
while condizione: # finché la condizione è vera...
    # corpo del while:
    # sequenza di istruzioni che vengono ripetute
    # finché condizione è vera
# NEXT INSTRUCTION
```

- La `condizione` viene valutata
- Se è vera, viene eseguito il corpo e la `condizione` viene rivalutata
- Se è falsa, si esce dal ciclo: il controllo passa a `# NEXT INSTRUCTION`

# Condizione iniziale e loop infinito

- Condizione inizialmente `False`: il corpo non viene mai eseguito
- Condizione inizialmente `True` e corpo che non la altera: **loop infinito**
- Il corpo del `while` deve poter **alterare la condizione**

```
while True: # finché la condizione è vera...
    print('fermatemiiii!')
```

Per interrompere: **Ctrl+C** nel terminale, oppure il pulsante di stop (o interrupt del kernel) dell'ambiente di sviluppo

# Soluzione: la tabellina con while

```
# i = 1
# finché i è minore di 10 esegui le seguenti istruzioni:
#   calcola 3*i
#   stampa il risultato
#   incrementa i
i = 1
n_tabellina = 3
while i <= 10:
    risultato = n_tabellina * i
    print(n_tabellina, '*', i, '=', risultato)
    i = i + 1
```

# Schemi logici dei cicli while

Prima di scrivere il codice conviene ragionare sulla **logica della ripetizione**: molti problemi si riconducono a pochi **schemi ricorrenti**

```
inizializzazione
while condizione:
    corpo del ciclo
    # durante il ciclo devono essere aggiornate
    # le variabili da cui dipende la condizione
```

Il corpo del ciclo viene eseguito finché la condizione rimane vera

# Due modi di controllare il ciclo

| Criterio | Idea |
|---|---|
| Numero di iterazioni **noto** in anticipo | ripetere un'operazione un numero noto di volte |
| Numero di iterazioni **non noto** in anticipo | ripetere finché una condizione, aggiornata a ogni iterazione, rimane True |

# Iterazioni note e non note

- Iterazioni note
- Il numero di ripetizioni si conosce prima del ciclo
- Serve un contatore: inizializzarlo, verificare il limite, aggiornarlo

- Iterazioni non note
- Il ciclo continua finché la condizione resta vera
- Conta l'evoluzione dello stato: a ogni iterazione si aggiornano le variabili

# Iterazioni note: il contatore

Un **contatore** viene incrementato a ogni iterazione e usato nella condizione

```
print('quanti numeri vuoi inserire?')
n = int(input())
i = 0 # contatore delle iterazioni
while i < n:
    i = i + 1 # i viene incrementata
    # i += 1 # sintassi abbreviata
    valore = float(input())
    print('hai inserito il numero ', valore)
print('Istruzione fuori dal ciclo')
```

# Come evolve il contatore con n == 3

| Iterazione | i all'inizio | i alla fine | i < n |
|---|---|---|---|
| 1 | 0 | 1 | True |
| 2 | 1 | 2 | True |
| 3 | 2 | 3 | False: si esce |

La condizione `i < n` viene valutata prima del ciclo (`i == 0`) e all'inizio di ogni iterazione: si eseguono esattamente `n` iterazioni

# Errori comuni

```
n = int(input()) # dimenticare int()
i = 0 # dimenticare di inizializzare i
while i < n:
    # sbagliare la condizione: i<=n, i>n, i != n
    i += 1 # dimenticare di incrementare
    # ...
```

# Iterazioni non note: la condizione

Il blocco viene ripetuto finché una condizione, valutata a ogni iterazione, rimane vera

```
while condizione:
    # istruzioni ripetute finché la condizione è vera
    # aggiornamento delle variabili della condizione
```

Non appena la condizione diventa `False`, il ciclo si interrompe

# Esempio

Il programma chiede dei valori all'utente.
Si ferma se l'utente inserisce 0.

**Soluzione:** chiedere il primo valore prima che il ciclo inizi

```
valore = int(input())
while valore != 0:
    print('hai inserito', valore)
    valore = int(input())
print('Istruzione fuori dal ciclo')
```

- Il primo valore si legge prima del ciclo
- Se è 0, il ciclo non viene mai eseguito
- Inserendo 0 la condizione diventa `False`
- Il valore 0 non viene stampato

# Esempio

Il programma chiede dei valori all'utente.
Si ferma se l'utente inserisce 0.

**Soluzione:** assegnare un valore diverso da 0 per forzare l'ingresso nel ciclo.

```
valore = 1
while valore != 0:
    valore = int(input())
    print('hai inserito', valore)
print('Istruzione fuori dal ciclo')
```

Differenze rispetto all'esempio precedente:
ora `print` viene eseguito anche nell'ultima iterazione, quando `valore == 0`

# Valore iniziale: meglio None

- **Attenzione:** assegnare un valore `!= 0` solo per forzare l'ingresso nel ciclo può causare confusione
- Meglio un valore diverso da tutti i possibili input: `valore = None`
- `None` rappresenta l'**assenza di valore** (oggetto nullo)

# Prima lettura fuori o dentro il ciclo

**Schema A**: prima lettura fuori dal ciclo

```
leggi valore
while valore != 0:
    elabora valore
    leggi valore
```

**Schema B**: prima lettura dentro il ciclo

```
inizializza valore
while valore != 0:
    leggi valore
    elabora valore
```

I due cicli sono **strutturalmente diversi**

# Esercizio: elenco di importi

- Scrivere un ciclo che chieda all'utente un elenco di importi
- Il ciclo termina quando l'utente inserisce un valore negativo
- Risolvere in due modi:
  - prima lettura **fuori** dal ciclo
  - prima lettura **dentro** il ciclo

# Variabile di controllo del ciclo

Dato che `while` valuta una condizione, la condizione può essere memorizzata in una variabile booleana, detta **variabile di controllo del ciclo**

```
continua = True # variabile di controllo
while continua: # cioè: continua == True
    valore = int(input())
    print('hai inserito', valore)
    continua = (valore != 0)
```

`valore != 0` produce un booleano (`True` o `False`), assegnato a `continua`: decide se il ciclo prosegue

# Assegnare direttamente un booleano

La riga `continua = (valore != 0)` equivale a:

```
if valore == 0:
    continua = False
else:
    continua = True
```



sAIfer Lab

Joint lab on Safety and Security of AI

# Esempi

Controllo del ciclo e obiettivo applicati a problemi concreti

# Media dei voti, numero di esami noto

## Iterazioni **note** · obiettivo: **accumulazione**

```
print('Quanti esami hai sostenuto?')
n = int(input())
i = 0 # contatore
s = 0 # accumulatore
while i < n:
    print('inserisci il voto')
    v = int(input())
    s = s + v
    i += 1  # equivale a i = i + 1
print('sono uscito dal ciclo')
media = s / n
print('media = ', media)
```

# Media pesata: iterazioni note, accumulazione

```
print('Quanti esami hai sostenuto?')
n = int(input())
i = 0
somma_voti_per_crediti = 0 # accumulatore
somma_crediti = 0          # accumulatore
while i < n:
    print('inserisci il voto')
    v = int(input())
    print('inserisci i crediti')
    c = int(input())
    somma_crediti = somma_crediti + c
    somma_voti_per_crediti = somma_voti_per_crediti + v*c
    i += 1
print('sono uscito dal ciclo')
media_pesata = somma_voti_per_crediti / somma_crediti
print('media pesata = ', media_pesata)
```

# Spesa totale: stop con 0

Iterazioni **non note** · obiettivo: **accumulazione**

```
s = 0 # accumulatore
print('inserisci il primo importo')
v = int(input())
while v != 0:
    s = s + v
    print('inserisci importo; 0 per uscire')
    v = int(input())
print('sono uscito dal ciclo')
print('spesa totale = ', s)
```

# Spesa totale: stop con un carattere

Solo importi interi · iterazioni **non note** · **accumulazione**

```
s = 0
print('inserisci il primo importo (solo cifre),',
      'oppure una lettera per terminare')
v = input() # non converto ancora la stringa in intero
while v.isdigit():
    s = s + int(v)
    print('inserisci importo (solo cifre),',
          'oppure una lettera per terminare')
    v = input()
print('sono uscito dal ciclo')
print('spesa totale = ', s)
```

# Il metodo isdigit()

- `v.isdigit()` rende `True` se la stringa `v` contiene solo cifre, cioè un intero senza segno: per es. `"45"`
- Rende `False` con `"-5"`, `"4.5"` o la stringa vuota `""`
- Vedremo in seguito come gestire le stringhe che rappresentano `float`

# Spesa totale: soluzione alternativa

```
s = 0
continua = True
while continua:
    print('inserisci importo (solo cifre),',
          'oppure una lettera per terminare')
    v = input()
    if v.isdigit():
        s = s + int(v)
    else:
        continua = False
print('sono uscito dal ciclo')
print('spesa totale = ', s)
```

# Stop con x: il problema

- Si inseriscono interi; si termina con il carattere **x**
- Altri caratteri vengono ignorati: il programma chiede un altro input e lo riesamina
- Iterazioni **non note** · obiettivo adattabile a molti scopi

# Stop con x: soluzione errata

```
print("inserisci un numero intero oppure 'x'",
      "- altri caratteri non ammessi")
v = input()
if v != 'x':
    if v.isdigit():
        v = int(v)
    else:
        print("hai inserito un carattere non ammesso")
        v = input()
        # a questo punto dovrei ricontrollare se v è
        # digit oppure x in un ciclo potenzialmente infinito
```

# Stop con x: soluzione corretta

```
print("inserisci un numero intero oppure 'x'",
      "- altri caratteri non ammessi")
v = input()
while v != 'x':
    if v.isdigit():
        print(v)
    else:
        print("hai inserito un carattere non ammesso")
    v = input()
```

# Contare i positivi su 10 numeri

## Iterazioni note · conteggio di occorrenze

```
n = 10
i = 0
n_positivi = 0
while i < n:
    v = int(input())
    if v > 0:
        n_positivi += 1
    i += 1
print('hai inserito', n_positivi, 'numeri positivi')
```

# True e False valgono 1 e 0

In un'espressione matematica `True` e `False` valgono `1` e `0`: la struttura condizionale si può sostituire con un costrutto più conciso

```
#if v > 0:
#    n_positivi += 1
n_positivi += v > 0 # True o False
```

# Contare i voti sufficienti

## Stop con un carattere · conteggio di occorrenze

```
voto_sufficienza = 6
# 6 per scuole superiori; 18 per univ
n_sufficienti = 0
print('inserisci voti; carattere per terminare')
v = input()
while v.isdigit(): # è convertibile in intero?
    voto = int(v)
    n_sufficienti += voto >= voto_sufficienza
    # equivale a
    # if voto >= voto_sufficienza:
    #    n_sufficienti += 1
    v = input()
print('hai inserito ', n_sufficienti, 'voti sufficienti')
```

# Al massimo 10 voti, uscita anticipata con un carattere

```
n = 10
voto_sufficienza = 6
n_voti = 0
n_sufficienti = 0
print('Inserisci fino a', n, 'voti;',
      'un carattere non numerico per terminare')
v = input()
while v.isdigit() and n_voti < n:
    voto = int(v)
    if voto >= voto_sufficienza:
        n_sufficienti += 1
    n_voti += 1
    if n_voti < n:
        v = input()
print('Voti inseriti:', n_voti)
print('Voti sufficienti:', n_sufficienti)
```

# Una soluzione poco elegante

- Dopo 10 valori numerici `n_voti` vale `n`: grazie all'ultimo `if` non si chiede un valore oltre il decimo
- Il controllo `n_voti < n` è ripetuto nella condizione e nel corpo del `while`
- Il costrutto `break`, che vedremo fra poco, permette altre soluzioni

# Soluzione con una variabile booleana

```
n = 10
voto_sufficienza = 6
n_voti = 0
n_sufficienti = 0
continua = True
print('Inserisci fino a', n, 'voti;',
      'un carattere non numerico per terminare')
while continua:
    v = input()
    if v.isdigit():
        voto = int(v)
        if voto >= voto_sufficienza:
            n_sufficienti += 1
        n_voti += 1
        continua = n_voti < n
    else:
        continua = False
print('Voti inseriti:', n_voti)
print('Voti sufficienti:', n_sufficienti)
```

# Due modi di uscire dal ciclo

- La lettura avviene all'inizio di ogni iterazione: non serve `input()` prima del ciclo
- Carattere non numerico: il ramo `else` mette `continua = False`
- Decimo voto inserito: `continua = n_voti < n` diventa `False`
- In entrambi i casi il `while` termina senza chiedere altri valori

# Rilevazione di evento

- Ogni esercizio di conteggio si può trasformare in **rilevazione di evento**
- Al posto del contatore si usa una variabile `flag`
- Esempio: su 10 voti, verificare se almeno un voto è maggiore di 27

# Almeno un voto maggiore di 27?

```
n = 10
i = 0
soglia = 27
flag = False
print('Inserisci voti')
while i < n:
    voto = int(input())
    if voto > soglia:
        flag = True
    i += 1
if flag:
    print('almeno un voto è maggiore di ', soglia)
else:
    print('nessun voto è maggiore di ', soglia)
```

# Esercizi su accumulazione

- Con iterazioni note, non note, oppure massime con stop anticipato
- Somma totale, media aritmetica, pesata, armonica, varianza
- La **mediana** richiede di memorizzare e ordinare tutti i valori: servono strumenti come le liste, che vedremo più avanti

# Ricerca di estremo: il voto massimo

`voto_max` conserva il voto più alto incontrato finora

```
i = 0
n = 10
voto_max = 0
while i < n:
    voto = int(input())
    if voto > voto_max:
        voto_max = voto
    i += 1
print('il voto massimo è ...', voto_max)
```

Si parte da `0`; alla fine `voto_max` contiene il massimo tra tutti i voti inseriti

# Temperature: perché non partire da 0?

- Con temperature tutte negative, `temp_max = 0` darebbe un risultato sbagliato
- Si legge la prima temperatura prima del ciclo e la si usa per inizializzare `temp_max`
- A ogni iterazione la nuova temperatura si confronta con il massimo corrente

# Temperatura massima su 10 valori

```
i = 0
n = 10
# prima lettura
temperatura = float(input())
temp_max = temperatura
i += 1
while i < n:
    temperatura = float(input())
    if temperatura > temp_max:
        temp_max = temperatura
    i += 1
print('la temperatura massima è ...', temp_max)
```

# Aggiornamento iterativo

- Popolazione con tasso di crescita: dopo quanto tempo raddoppia?
- Tasso di decrescita: quando si dimezza, o scende sotto una soglia?
- Capitale con tasso di interesse: quanti anni per raddoppiarlo?

# Quanti anni per raddoppiare il capitale?

```
# data una cifra iniziale ed un tasso di interesse,
# quanti anni devono passare per raddoppiare la cifra?
interesse = float(input('inserisci il tasso di interesse '
                        '(es. 0.05 per 5%): '))
capitale_iniziale = int(input('inserisci capitale iniziale'))
obiettivo = 2 * capitale_iniziale
capitale_attuale = capitale_iniziale
anno = 0
while capitale_attuale < obiettivo:
    anno += 1
    capitale_attuale = capitale_attuale * (1 + interesse)
    print('anno', anno, 'ora hai', capitale_attuale)
    # stampa con due decimali:
    # print("anno", anno, f"ora hai {capitale_attuale:.2f}")
print('fine del ciclo: ora hai', capitale_attuale)
print('anni trascorsi', anno)
```



sAIfer Lab

Joint lab on Safety and Security of AI

# Sintassi completa: break, continue, else

Controllare l'esecuzione del ciclo

# break e continue

- `break`
- Interrompe il ciclo e passa all'istruzione dopo il `while`
- Nei cicli annidati (nested) interrompe solo il ciclo più interno

- `continue`
- Salta il resto dell'iterazione corrente e passa alla successiva
- Non interrompe il ciclo

# La clausola else

```
while condizione:
    # Blocco eseguito finché la condizione è vera
else:
    # Blocco eseguito solo se il ciclo termina
    # normalmente (senza break)
```

- Il corpo del `while` viene eseguito finché `condizione == True`
- Se il ciclo termina normalmente, viene eseguito il blocco `else`
- Se il ciclo viene interrotto con `break`, il blocco `else` non viene eseguito

docs.python.org/it/3.13/reference/compound_stmts.html#while

# Uscire dal ciclo: senza break

Esempio: chiedere numeri finché l'utente non digita 0

```
numero = int(input())
while numero != 0:
    print("Hai inserito:", numero)
    numero = int(input())
print("Hai scelto di uscire dal ciclo.")
```

# Uscire dal ciclo con break

Esempio: chiedere numeri finché l'utente non digita 0

```
while True:  # Un ciclo teoricamente infinito
    print("Inserisci un numero (0 per uscire): ")
    numero = int(input())
    if numero == 0:
        print("Hai scelto di uscire dal ciclo.")
        break  # Esce dal ciclo
    print("Hai inserito:", numero)
```

# Early stopping: senza break

Itera 10 volte, oppure esci prima se l'utente inserisce 0

```
i = 0
n = 10
v = int(input())
print(v)
i += 1
while i < n and v != 0:
    v = int(input())
    print(v)
    i += 1
print('sono fuori dal ciclo')
if v == 0:
    print('sono uscito perché hai inserito 0')
```

Questa versione stampa anche lo `0` prima di uscire

# Early stopping con break

Itera 10 volte, oppure esci prima se l'utente inserisce 0

```
i = 0
n = 10
while i < n:
    v = int(input())
    if v == 0:
        print('è stato inserito 0')
        break
    print(v)
    i += 1
print('sono fuori dal ciclo')
```

# La clausola else: un esempio

Il blocco `else` distingue i due motivi di uscita dal ciclo

```
i = 0
n = 10
while i < n:
    v = int(input())
    if v == 0:
        print('è stato inserito 0 - esco dal ciclo')
        break
    print(v)
    i += 1
else:
    print('Ciclo terminato perché sono state eseguite',
          'tutte le iterazioni richieste')
print('sono fuori dal ciclo')
```

# Cicli nidificati

## Un `while` dentro un altro: stampa tutti i 100 elementi

```
# ricavo gli indici degli elementi
# di una matrice quadrata 10 * 10
n = 10
i = 0
while i < n:
    j = 0
    i += 1
    while j < n:
        j += 1
        print("elemento", i, j)
```

```
elemento 1 1
elemento 1 2
...
elemento 1 10
elemento 2 1
...
elemento 10 10
```

# break nel ciclo esterno

Il `break` termina il ciclo in cui è inserito: qui il ciclo esterno

```
# ricavo gli indici degli elementi
# di una matrice quadrata 10 * 10
n = 10
i = 0
while i < n:
    j = 0
    i += 1
    if i == 2:
        break
    while j < n:
        j += 1
        print("elemento", i, j)
```

```
elemento 1 1
elemento 1 2
...
elemento 1 10
```

# break nel ciclo interno

Qui il `break` termina il ciclo **interno**: per `i == 2` non si stampa nulla

```
# ricavo gli indici degli elementi
# di una matrice quadrata 10 * 10
n = 10
i = 0
while i < n:
    j = 0
    i += 1
    while j < n:
        j += 1
        if i == 2:
            break
        print("elemento", i, j)
```

```
elemento 1 1
...
elemento 1 10
elemento 3 1
...
elemento 10 10
```

# Saltare un'iterazione con continue

`continue` salta l'iterazione corrente e passa alla successiva, senza interrompere il `while`

Esempio: stampare i numeri da 1 a 10, saltando i multipli di 3

```
numero = 0
while numero < 10:
    numero += 1
    if numero % 3 == 0:
        # se il resto della divisione tra numero e 3 è 0
        # salta il resto e passa alla prossima iterazione
        continue
    print(numero)
```



sAIfer Lab

Joint lab on Safety and Security of AI

# Esercizi proposti

Non in ordine di difficoltà

# Risolvere gli esercizi in più modi

- Con un numero massimo di iterazioni
- Con un criterio di stop
- Usando entrambi i criteri
- Usando (o non usando) `break` e `continue`
- Combinando due condizioni con operatori booleani

# Esercizi su medie e successioni

- Dato un intero, calcolare il fattoriale
- Media escludendo il voto più basso
- Media escludendo il voto più basso e il più alto (media robusta)
- Primi n elementi della successione di Fibonacci: F(n) = F(n-1) + F(n-2)

it.wikipedia.org/wiki/Successione_di_Fibonacci

# Esercizi su condizioni e stringhe

- Spesa finale con uno sconto legato, per esempio, al numero di acquisti o all'importo totale
- Sommare solo i valori che soddisfano una condizione
- Dato `n`, stampare i numeri pari fino a `n`: con `n == 10` si stampa `2, 4, 6, 8, 10`
- Acquisire una serie di frasi e stampare la più lunga (usare `len(stringa)`)

# Esercizio: crescita di una popolazione

- Calcolare gli individui di una popolazione noto il tasso di crescita; ogni iterazione è un anno
- Il programma termina quando la popolazione è raddoppiata o dimezzata (il tasso può essere negativo)
- Variante: l'utente aggiorna il tasso di crescita ogni anno

# Esercizio: somma con azzeramento

- Il programma chiede quanti valori (addendi) inserire
- L'inserimento termina con il carattere `x` o al raggiungimento del numero di addendi
- Con `-1` la somma corrente si azzera; `-1` non conta tra gli addendi

# Contare le cifre di un numero

Dato un intero positivo, contare le cifre manipolando il numero, non la stringa

```
# Soluzione
print("Inserisci un numero intero,",
      "calcolerò il numero di cifre")
n = int(input())
i = 0
while n >= 1:
    i = i + 1
    n = n // 10
print('Numero di cifre ---> ', i)
```

Con input `0` risponde 0 cifre: serve un controllo in più

# MCD e numeri primi

- Dati due interi positivi, trovare il massimo comune divisore (MCD) con l'algoritmo di Euclide
- Dire se p è un numero primo
- Varianti: dire se p è un primo di Sophie Germain o di Mersenne

it.wikipedia.org/wiki/Algoritmo_di_Euclide
it.wikipedia.org/wiki/Numero_primo_di_Sophie_Germain
it.wikipedia.org/wiki/Numero_primo_di_Mersenne

# Somma fino a 100, al massimo 10 inserimenti

```
print('Inserisci i dati, termino quando la somma',
      'eccede 100 o dopo 10 inserimenti!')
s = 0
i = 0
while s <= 100 and i < 10:
    # DE MORGAN
    # not(s>100 or i>=10)
    v = int(input())
    s = s + v
    i += 1
print('somma:', s)
print('numero inserimenti:', i)
# VARIANTE:
# stampare un messaggio che spieghi se il ciclo è terminato
# perché somma >100 o perché si sono superate
# le 10 iterazioni
```

# Base 2 → base 10, bit inseriti dal msb

```
# SOLUZIONE:
print("Preparati ad inserire un numero intero",
      "espresso in base 2, a partire da MSB")
print("Quanti bit?")
n = int(input())
print("Ok, mi preparo a convertire un numero di ", n,
      " bit. Comincia l'inserimento")
i = 0
v = 0
while i < n:
    v = v * 2
    b = int(input())
    v = v + b
    i += 1
print('In base 10: ---> ', v)
```

# Stampare un numero al contrario

Manipolare il numero, **non** la sua stringa

```
# Soluzione
n_reverse = 0
print('Inserisci un numero')
n = int(input())
while n >= 1:
    resto = n % 10
    n = n // 10
    n_reverse = n_reverse * 10
    n_reverse = n_reverse + resto
print(n_reverse)
```

Gli zeri finali si perdono: `1230` diventa `321`, perché come numero `0321` vale `321`

# Indovina il numero segreto

- Il computer sceglie un numero segreto tra 1 e 100; l'utente ha 5 tentativi
- Dopo ogni tentativo il programma dice “Troppo alto” o “Troppo basso”
- Si termina quando l'utente indovina o esaurisce i tentativi
- Suggerimento: usa la funzione `random.randint()`

# Indovina il numero: soluzione

```
import random
# genera un numero (pseudo) casuale tra 1 e 100
numero_segreto = random.randint(1, 100)
n = 5
i = 0
while i < n:
    i = i + 1
    print('indovina il numero')
    v = int(input())
    if v == numero_segreto:
        print('INDOVINATO!')
        break
    elif v < numero_segreto: # riferito a IF
        print('Hai inserito un numero troppo piccolo')
    else:
        print('Hai inserito un numero troppo grande')
else: # riferito a WHILE
    print('mi dispiace, hai esaurito i tentativi')
    print('il numero era ', numero_segreto)
```

# Sfida: senza la clausola else

- La maggior parte dei linguaggi di programmazione ha il `while` **senza** la clausola `else`
- Formulate una soluzione alternativa senza l'`else` del `while`, implementabile anche in altri linguaggi